

03 — Relational Databases

Key Concepts

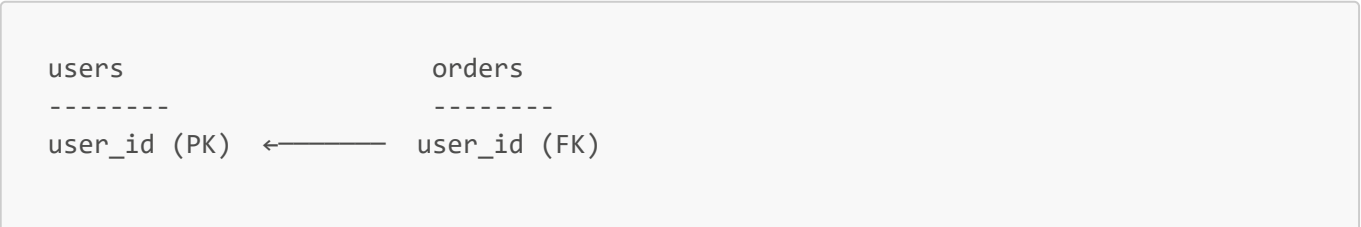
Term	Definition
Table	Data stored in rows (records) and columns (fields)
Primary Key (PK)	Uniquely identifies each row in a table
Foreign Key (FK)	Column referencing another table's PK — defines a relationship
Schema	Blueprint of the database — tables, columns, types, constraints
Index	Data structure that speeds up column lookups — avoids full table scan
Transaction	A unit of work that must complete fully or not at all
Query	SQL statement to read or manipulate data

ACID Properties

Property	Meaning	Example
Atomicity	All-or-nothing — no partial state	Bank transfer: debit + credit both succeed or neither does
Consistency	Constraints always enforced — valid state to valid state	Can't insert order for non-existent user
Isolation	Concurrent transactions don't interfere	Two users booking last seat — only one succeeds
Durability	Committed data survives crashes — written to disk	Data safe even if server goes down after commit

Relationships

Type	Example	Implementation
One-to-One	User → Profile	FK in either table
One-to-Many	User → Orders	<code>user_id</code> FK on the orders table
Many-to-Many	Students ↔ Courses	Junction table (e.g. <code>enrollments</code> with both FKs)



name	order_id (PK)
email	amount

JOIN Types

JOIN	Returns
INNER JOIN	Only rows that match in both tables
LEFT JOIN	All rows from left table + matching right rows (NULL if no match)
RIGHT JOIN	All rows from right table + matching left rows (NULL if no match)
FULL OUTER JOIN	All rows from both tables (NULL where no match)
CROSS JOIN	Every combination of rows from both tables

```
-- INNER JOIN: only matched rows
SELECT u.name, o.amount
FROM users u
INNER JOIN orders o ON u.user_id = o.user_id;

-- LEFT JOIN: all users, even those with no orders
SELECT u.name, o.amount
FROM users u
LEFT JOIN orders o ON u.user_id = o.user_id;
```

When to use LEFT JOIN: When the left table has primary data and right-side matches may be absent but you still need all left rows.

SQL Basics

```
SELECT * FROM users WHERE age > 25 ORDER BY name;
INSERT INTO users (name, email) VALUES ('Anil', 'a@b.com');
UPDATE users SET email = 'new@b.com' WHERE user_id = 1;
DELETE FROM users WHERE user_id = 1;
```

Database Comparison

Database	Best for	Used by
PostgreSQL	Complex queries, JSON, strict standards	Instagram, Reddit, Shopify
MySQL	High read throughput, web apps	Facebook (historically), WordPress

Database	Best for	Used by
SQLite	Embedded, zero-config, single file	Mobile apps, browsers, small tools
SQL Server	Microsoft/enterprise ecosystem	Banks, .NET shops

PostgreSQL vs MySQL: PostgreSQL is feature-richer (JSON, window functions, full-text search). MySQL historically faster for simple reads. PostgreSQL is the default choice for new projects today.

Industry Examples

Company	Usage
Instagram	PostgreSQL at scale, sharded across machines
Shopify	MySQL with heavy indexing for Black Friday traffic
SQLite	Inside every Chrome browser, every iOS/Android device
GitHub	MySQL with read replicas to handle traffic

Interview Questions

Q: What is a relational database? A: Stores data in tables where rows relate to each other via foreign keys. Avoids data duplication — reference once, join when needed.

Q: Explain ACID. A: Atomicity (all-or-nothing), Consistency (constraints always valid), Isolation (concurrent transactions don't interfere), Durability (committed data survives crashes).

Q: What is the difference between INNER JOIN and LEFT JOIN? A: INNER JOIN returns only rows matching in both tables. LEFT JOIN returns all rows from the left table with NULLs where no match exists in the right.

Q: How do you model a one-to-many relationship? A: Add a foreign key column on the "many" side. E.g. `orders.user_id` references `users.user_id` — one user, many orders.

Q: Why would you choose PostgreSQL over MySQL? A: PostgreSQL has richer features (JSON, window functions, full-text search), stricter SQL standards compliance, and is the default choice for most new projects today.